

An abstract graphic of glowing green lines and dots on a dark background, resembling a circuit board or a network diagram. The lines are thin and sharp, with some dots at the intersections. The overall style is futuristic and technical.

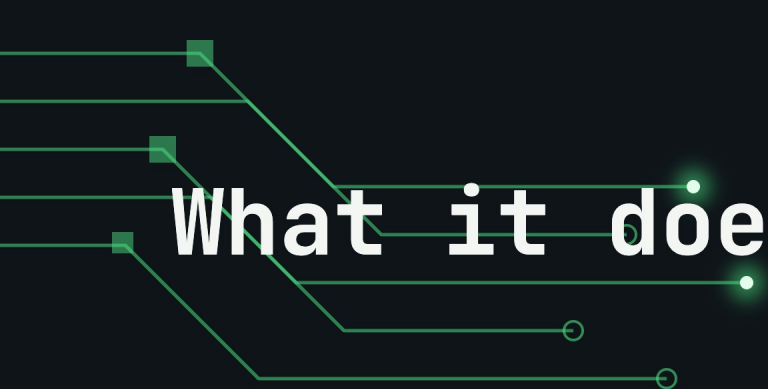
```
$ ./bank-of-cli
```

# Bank of CLI

A terminal banking application with a persistent ledger, atomic transfers, and a layered architecture.

---

By Ethan Duarte



# What it does

01

## Secure access

Register and log in with a generated account ID and a 4-digit PIN.

02

## Transaction engine

Deposit, withdraw, and transfer with overdraft protection.

03

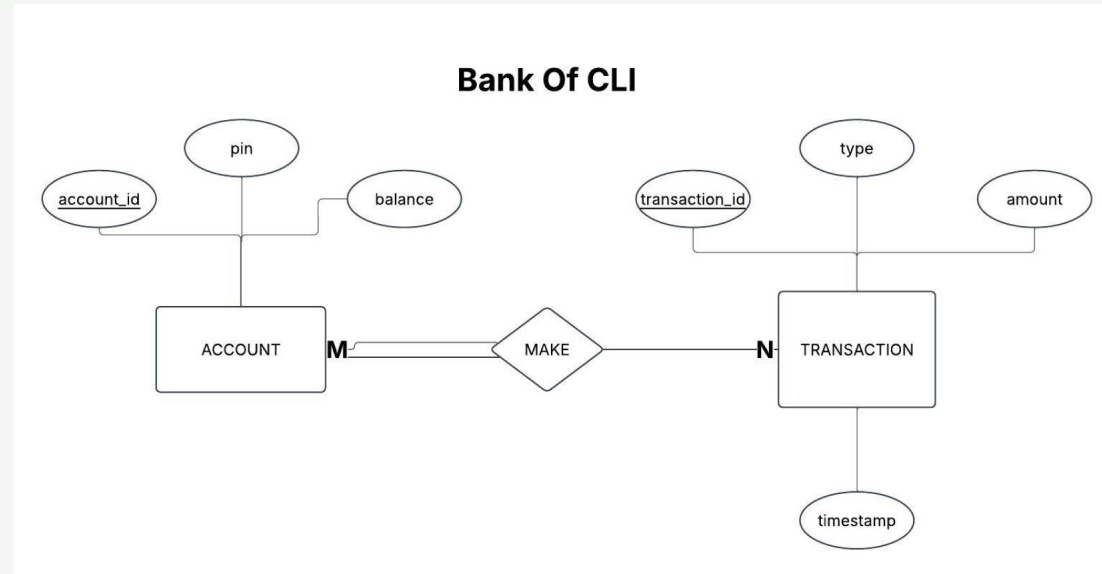
## Audit trail

Every movement of money is recorded and queryable per account.



Everything runs in the terminal, backed by a relational database

# The data model



One account makes many transactions. A transfer writes **two** rows: (Row 1 - Sender, Row 2 - Receiver) — linked by **related\_account\_id**.

# The 4 Layers

CALLS  
DOWN

**api**

Terminal I/O, menus, and input validation. Talks only to the service layer.

**service**

Banking rules: PIN format, positive amounts, sufficient funds, no self-transfer.

**persistence**

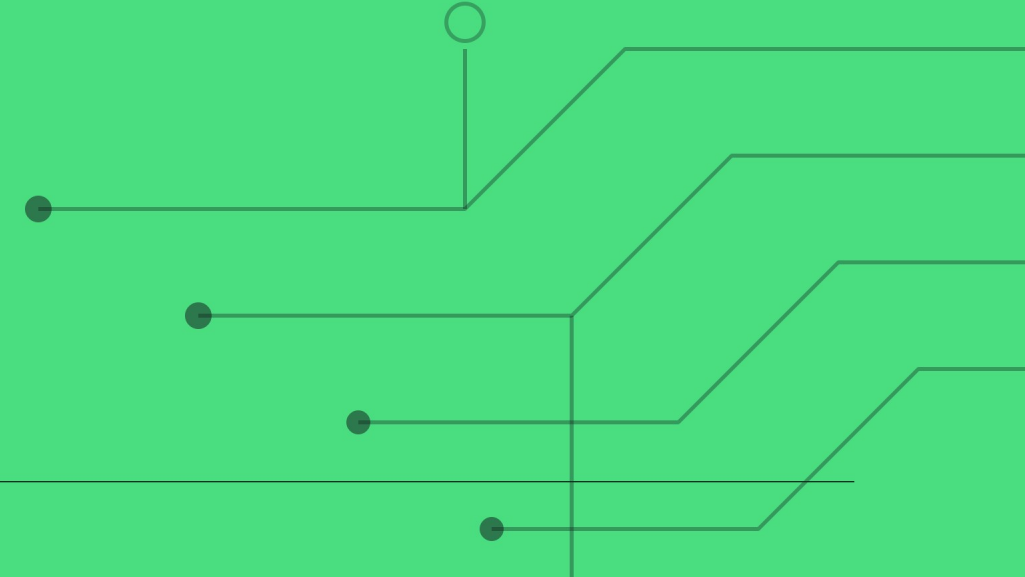
JDBC, SQL, and transaction control. Maps result set rows to Java objects.

**domain**

Account and Transaction. Every layer depends on it; it depends on nothing.

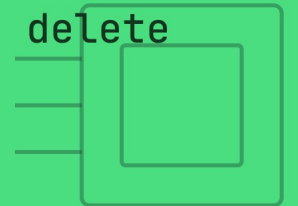
This is the **DAO pattern**: all data access sits behind a **BankDAO** interface, so the service layer never touches SQL.

LIVE DEMO

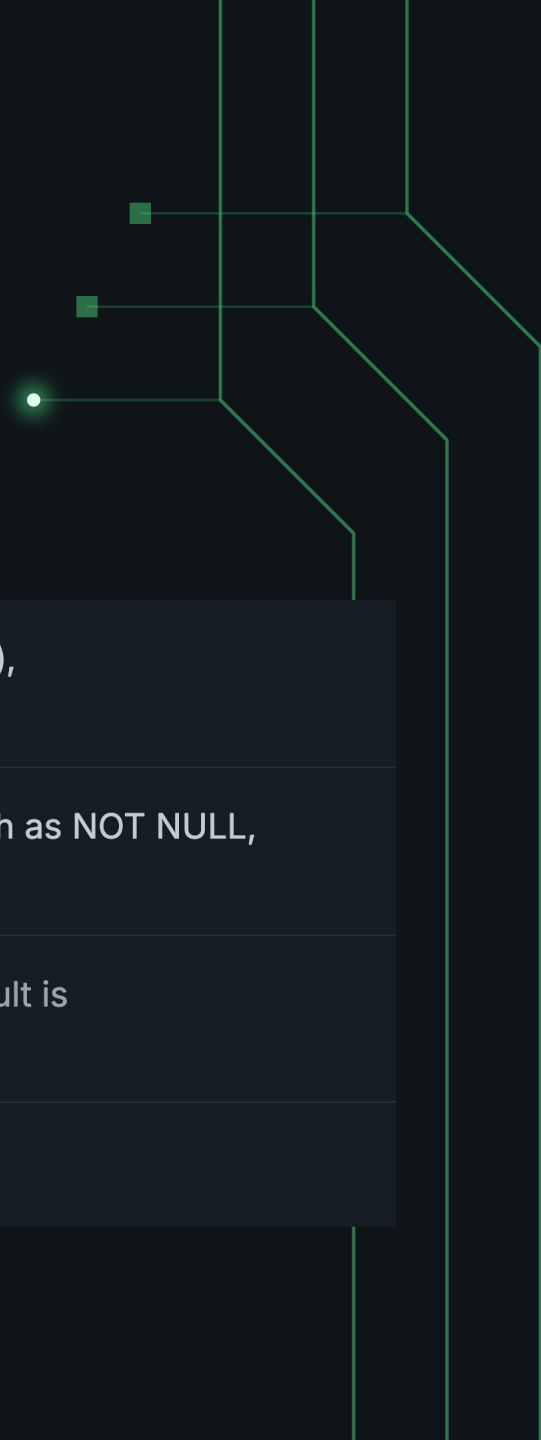
A series of thin, dark grey lines forming a circuit-like pattern in the top right corner. The lines are horizontal and vertical, with some diagonal segments. Small black dots are placed at various points along these lines, resembling solder points or connection nodes. The pattern starts from the right edge and extends towards the center of the top half of the image.

# Bank Of CLI Demo

signup → login → deposit → withdraw → transfer → transactions → update → delete



# Where ACID Lives



## Atomicity

All four statements in a transfer land together or not at all : **setAutoCommit(false), commit(), rollback()**.

## Consistency

The constraints defined when the entities were created hold before and after such as NOT NULL, foreign keys, NUMERIC(12,2), and **AND balance >= ?**.

## Isolation

Concurrent transfers don't see each other's half-finished work. Postgres by default is **READ COMMITTED**.

## Durability

Once committed, it survives a crash. Handled by the Postgres write-ahead log.



# Testing

## NEGATIVE TEST

### A self-transfer must be rejected

**assertThrows** proves the service throws when the source and destination are the same account — and it passes a null DAO, so the rule is enforced before the database is ever touched.

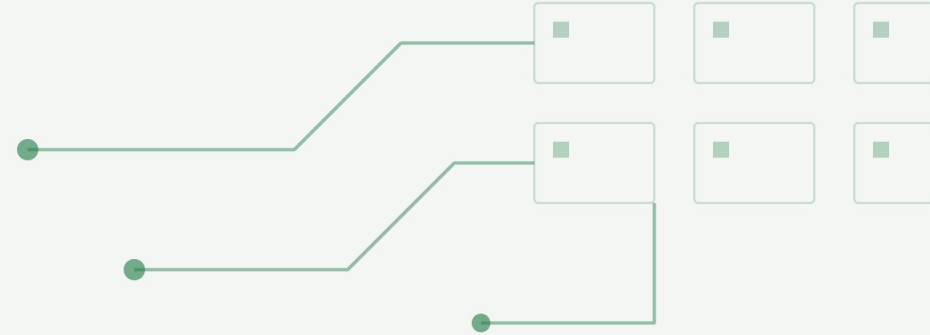
## POSITIVE TEST

### A valid transfer moves the money

Two accounts, one transfer of 200, then **assertEquals** on both balances — 300 out of one, 200 into the other.

Both follow **Arrange, Act, Assert**. The positive test still hits the real database, because **BankServiceImpl** takes the DAO as an interface.

# Technologies used



## Java 17

The language, with switch expressions, lambdas, and try-with-resources.

## Maven

Build tool and dependency management via the POM.

## PostgreSQL

The relational database holding accounts and the transaction ledger.

## JDBC

The Java API connecting the application to that database.

## SLF4J + Logback

The logging interface I code against, and the implementation that writes a rolling daily file.

## JUnit 5

Unit testing for the service layer.

# What I'd do next



## BigDecimal

Balances are NUMERIC in Postgres but double in Java. Floating point has no place in money.

---

## Hashed PINs

PINs are stored in plaintext today. BCrypt with a per-account salt is the fix.

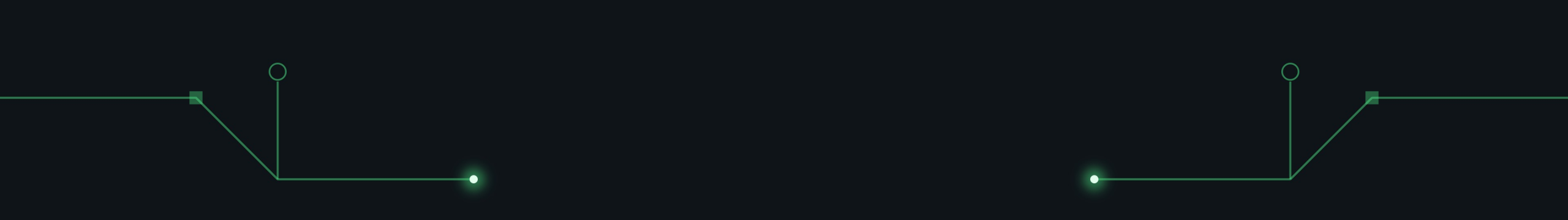
---

## Stubbed tests

Swap the real DAO for a stub so service tests run without a live database.

[github.com/aduart03/BANK-OF-CLI](https://github.com/aduart03/BANK-OF-CLI)





# Q & A

---

[github.com/aduart03/BANK-OF-CLI](https://github.com/aduart03/BANK-OF-CLI)

